

NovaCampus Alliance – Git, GitHub and Versioning Guide

Note: Every example in this guide (branch names, commit messages, feature descriptions, and version numbers) is purely random and **does not** represent any real feature, user story or technical decision of the NovaCampus Alliance project.

1. Branching Strategy & GitHub Workflow

We follow a lightweight Git Flow adapted to SCRUM sprints. It keeps **main** clean while allowing parallel work on features and the AI agent.

Protected Branches

Branch	Purpose	Protection rules
main	Stable code only. Used for tags and demos	Require PR + 1 approval, no force-push
develop	Current sprint integration branch	Require PR + 1 approval

Branch Naming Rules

Format: `type/<issue-number>-<kebab-case-short-description>`

Examples:

- `feature/15-student-timetable-view`
- `feature/42-ai-room-allocation-agent`
- `fix/67-grade-calculation-error`
- `spike/89-rag-query-performance` (only for short research spikes)

Recommended Daily Workflow

```
# 1. Update your local develop
git checkout develop
git pull origin develop

# 2. Create feature branch from develop
git checkout -b feature/15-student-timetable-view

# 3. Work with small, conventional commits
git commit -m "feat(timetable): add real-time room change notifications"

# 4. Push and open Pull Request
git push -u origin feature/15-student-timetable-view
# → Open PR on GitHub, link issue #15, request review from teammate
```

Both the terminal commands shown above and VS Code can be used. The examples use the command line for clarity and reproducibility, but you can perform every operation directly in VS Code:

- Use the **Source Control** view (left sidebar) to stage changes, write conventional commit messages and push.
- Install the **GitHub Pull Requests and Issues** extension to create branches from issues, open pull requests, link them to the SCRUM board and request reviews without leaving the editor.

After approval the branch is merged into `develop`.

Merging `develop` into `main` (Sprint / Milestone release)

This operation is **not** done after every feature. It is performed at the end of a sprint or when the team wants to create a new milestone version.

The choice of always going through a Pull Request (even from `develop` to `main`) guarantees that `main` stays protected and that the team has visibility on what is being released.

Steps:

1. Make sure `develop` is up-to-date and all tests / Docker Compose checks pass.
2. Go to GitHub → create a **Pull Request** from `develop` → `main`.
3. Give it a clear title, for example: `Release: Sprint 3 + AI scheduling agent MVP`.
4. Link it to the relevant sprint milestone or GitHub Project.
5. At least one team member reviews the changes (even if it is mostly a summary of merged features).
6. Merge the Pull Request (use "Create a merge commit" or "Squash and merge" depending on team preference).
7. Immediately after the merge, create and push an annotated version tag on `main`:

```
git checkout main
git pull origin main
git tag -a v0.3.0-sprint3 -m "Sprint 3 milestone: AI agent integrated + improved student frontend"
git push origin v0.3.0-sprint3
```

After this step, `main` contains the new stable version and the tag is ready for Docker images or the presentation demo.

GitHub Configuration

- Work inside the GitHub Organization `PROJETWEB-CESI` and the private repository `Project-Web-Group1`
- Turn on **branch protection** on both `main` and `develop`
- Use **GitHub Projects** as the SCRUM board (columns: Backlog, Sprint Backlog, In Progress, In Review, Done)
- Add a `CODEOWNERS` file later (example: `/services/ai-agent/*` owned by the AI team member)
- Use issue templates and a PR template for consistency

2. Semantic Versioning (SemVer)

Format: **MAJOR.MINOR.PATCH**[-pre-release]

The choice of Semantic Versioning makes every number immediately meaningful to the whole team and to anyone reading the Git history or Docker images.

What each number means

Number	When to increment	NovaCampus example	Resets the lower numbers?
MAJOR	Breaking change (API contract, DB schema, tool interface)	Changing the MCP tools exposed by the AI agent	Yes (MINOR & PATCH → 0)
MINOR	New backward-compatible feature	Adding AI-powered room conflict detection to the scheduling service	Yes (PATCH → 0)
PATCH	Bug fix or small non-breaking improvement	Fixing an overlap calculation bug in the timetable microservice	No

Pre-release suffix (-something)

Use it to mark development stages or sprint deliverables:

- **0.2.0-sprint3** → end of sprint 3
- **0.3.0-ai-mvp** → first working version of the AI agent integrated in a business process
- **1.0.0-rc.1** → release candidate before the final presentation

How to create and push a version tag

```
# Create annotated tag
git tag -a v0.2.0-sprint3 -m "Sprint 3 milestone: AI scheduling agent MVP + student frontend v2"

# Push the tag
git push origin v0.2.0-sprint3
```

Starting versions for this project

Because the ERP is still under development, we begin in the **0.x.y** range:

- **0.1.0** → after the modeling deliverable + first working core services and student frontend
- **0.2.0-sprintX** → after each sprint that delivers visible progress (especially AI agent features)
- We stay below **1.0.0** until the system is considered complete for the academic presentation.

Docker Compose example

```
services:
  api-gateway:
    image: ghcr.io/novacampus-alliance/api-gateway:0.2.0-sprint3
  ai-agent:
    image: ghcr.io/novacampus-alliance/ai-agent:0.2.0-ai-mvp
```